# Trillium Quickstart

## Contents

**Overview**

**Getting started on Trillium**
- Logging in
- Storage
- Software
- Tips for loading software
- Using commercial software

**Testing and debugging**

**Submitting jobs to the scheduler**
- Trillium specific restrictions
  - Job output must be written to the scratch file system
  - Default scheduler account
  - No job submission from jobs
  - Whole node or whole gpu scheduling
  - Memory requests are ignored
- Common options for job script
- Submitting jobs on the CPU subcluster
  - Partitions and limits
  - Example: MPI job
  - Example: OpenMP job
  - Example: hybrid MPI/OpenMP job
- Submitting jobs for the GPU subcluster
  - Partitions and limits
  - Example: Single-GPU Job
  - Example: Whole-Node (4 GPUs) Job
  - Example: Multi-Node GPU Job
  - Best Practices for GPU Jobs

**Monitoring**
- Monitoring the queue
- Monitoring running and past jobs

**Quick Reference for Common Commands**

# Overview

Trillium is a large parallel cluster built by Lenovo Canada and hosted by SciNet at the University of Toronto. It consists of three main components:

1. CPU Subcluster

- 235,008 cores provided by 1224 CPU compute nodes
- Each CPU compute node has 192 cores from two 96-core AMD EPYC 9655 CPUs ("Zen 5" a.k.a. "Turin") at 2.6 GHz (base frequency)
- Each CPU compute node has 755 GiB / 810 GB of available memory

- The nodes are connected by a non-blocking (1:1) 400 Gb/s InfiniBand NDR interconnect
- This subcluster is designed for large-scale parallel workloads

2. GPU Subcluster

- 252 GPUs provided by 63 GPU compute nodes.
- Each GPU compute node has 4 NVIDIA H100 (SXM) GPUs with 80 GB of dedicated VRAM
- Each GPU compute node also has 96 cores from one 96-core AMD EPYC 9654 CPUs ("Zen 4" a.k.a. "Genoa") at 2.4 GHz (base frequency)
- The nodes are connected by a non-blocking (1:1) 800 Gb/s InfiniBand NDR interconnect, i.e. 200 Gb/s per GPU
- Has a dedicated login node (trig-login01) with 4 four NVIDIA H100 (SXM) GPUs.
- This subcluster is optimized for AI/ML and accelerated science workloads

3. Storage System

- Unified 29 PB VAST NVMe storage for all workloads
- All flash-based for consistent performance
- Accessible as a standard shared parallel file system.

# Getting started on Trillium

You need an active **CCDB (https://ccdb.alliancecan.ca)** account from the **Digital Research Alliance of Canada (https://alliancecan.ca/en)**. With that, you can then request access to Trillium on the **Access Systems (https://ccdb.alliancecan.ca/me/access_systems)** page on the **CCDB (https://ccdb.alliancecan.ca)** site. After clicking the "I request access" button, it usually takes about an hour for your account to be actually created and available on Trillium.

Please read this present document carefully. The **Frequently Asked Questions** is also a useful resource. If at any time you require assistance, or if something is unclear, please do not hesitate to **contact us (mailto:trillium@tech.alliancecan.ca)**.

## Logging in

There are two ways to access Trillium:

1. Via your browser with Open OnDemand. This is recommended for users who are not familiar with Linux or the command line. Please see our quickstart guide (https://docs.scinet.utoronto.ca/index.php/Open_OnDemand_Quickstart) for more instructions on how to use Open OnDemand.
2. Terminal access with ssh. Please read the following instructions.

As with all SciNet and Alliance compute systems, access is done via **SSH** (secure shell). Furthermore, for Trillium specifically, authentication is only allowed via SSH keys that are uploaded to the **CCDB (https://ccdb.alliancecan.ca)**. **Please refer to this page** on how to generate your SSH key pair, upload, and use SSH Keys.

Trillium runs Rocky Linux 9.6, which is a type of Linux. You will need to be familiar with the Linux shell to work on Trillium. If you are not, it will be worth your time to review the **Linux introduction**, to attend a **Linux Shell course (https://explora.alliancecan.ca/events?include_expired=true&keywords=Shell)**, or to take some of our **Self-paced courses**.

You can use **SSH** by opening a terminal window (e.g. **Connecting with PuTTY** on Windows or **Connecting with MobaXTerm**), then SSH into the Trillium login nodes with your CCDB credentials.

- Use this command to log into one of the login nodes of the CPU subcluster:

```
$ ssh -i /PATH/TO/SSH_PRIVATE_KEY  MYALLIANCEUSERNAME@trillium.scinet.utoronto.ca
```

- To log into the login node for the GPU cluster, use this command

```
$ ssh -i /PATH/TO/SSH_PRIVATE_KEY  MYALLIANCEUSERNAME@trillium-gpu.scinet.utoronto.ca
```

Here, `/PATH/TO/SSH_PRIVATE_KEY` is the path to your private SSH key and `MYALLIANCEUSERNAME` is your username on the CCDB.

**Note that:**

- The Trillium login nodes are where you develop, edit, compile, prepare and submit jobs.
- The CPU login nodes and the GPU login node are not part of the compute nodes but they have the same architecture, operating system, and software stack as the CPU and GPU compute nodes, respectively.
- You can ssh from one login node to another using their internal hostnames `tri-login01, ..., tri-login06` and `trig-login01` (the latter is the GPU login node).
- If you add the option -Y you enable X11 forwarding, which allows graphical programs on Trillium to open windows on your local computer.
- To run on compute nodes, you must submit a batch job.

**On the login nodes, you may not:**

- Run large memory jobs
- Run parallel training or highly multi-threaded processes
- Run long computations (keep them under a few minutes)
- Run resource-intensive tasks like I/O-heavy operations or simulation.

If you cannot log in, be sure to first check the **System Status (https://status.alliancecan.ca)**, ensure your **CCDB (https://ccdb.alliancecan.ca)** account is active and that your public key was uploaded (in openssh format) to CCDB, and check that you had requested access on the **Access Systems (https://ccdb.alliancecan.ca/me/access_systems)** page.

# Storage

Trillium features a unified high-performance storage system based on the VAST platform. It serves the following directories:

- `home file system` – For personal files and configurations.
- `scratch file system` – High-speed, temporary personal storage for job data.
- `project file system` – Shared storage for project teams and collaborations.

For your convenience, the location of the top level of your home and scratch directories on these file systems are available in the environment variables `$HOME` and `$SCRATCH`, while the variable `$PROJECT` points at your directory on /project.

You may be part of several projects. In that case, `$PROJECT` points at your last project in alphabetical order (often, that is the one associated with an allocation). But you can find all the top level directories of projects that you have access to in `$HOME/links/projects`, next to a link `$HOME/links/scratch` which points to `$SCRATCH`. If you do not see the directory `$HOME/links` in your account, you can get it by running the command

```
$ trisetup
```

The content of the `$HOME/links/projects` will automatically update when you leave or join projects.

On **HPSS**, the nearline system to be attached to Trillium, there will also be an environment variable called `$ARCHIVE` to point at the location of your top directory there, if you have one.

The table below summarized the available space and policiies for each location:

| location | quota | expiration time | backed up | on login nodes | on compute nodes |
|---|---|---|---|---|---|
| $HOME | 100 GB per user | none | yes | yes | read-only |
| $SCRATCH | 25 TB per user(1) | TBD* | no | yes | yes |
| $PROJECT | determined by RAC allocation<br>1 TB per default group(2) | none | yes | yes | read-only |
| $ARCHIVE | determined by RAC allocation(2) | none | dual-copy | no | no |

(1)The SCRATCH policies are still subject to revision.

(2)There is no RAC mechanism to increase project ($PROJECT) and nearline ($ARCHIVE) quotas on Trillium.

# Software

Trillium uses the **environment modules** system to manage compilers, libraries, and other software packages. Modules dynamically modify your environment (e.g., `PATH`, `LD_LIBRARY_PATH`) so you can access different versions of software without conflicts.

Commonly used module commands:

- `module load <module-name>` – Load the default version of a software package.
- `module load <module-name>/<module-version>` – Load a specific version.
- `module purge` – Unload all currently loaded modules.
- `module avail` – List available modules that can be loaded.
- `module list` – Show currently loaded modules.
- `module spider` or `module spider <module-name>` – Search for modules and their versions.

Handy abbreviations are available:

- `ml` – Equivalent to `module list`.
- `ml <module-name>` – Equivalent to `module load <module-name>`.

When you have just logged in, only the `CCconfig`, `gentoo/2023` and `mii` modules are loaded, which provide basic OS-level functionality. To get a standard set of compilers and libraries like on the other compute clusters in the Alliance, you load the `StdEnv/2023`.

# Tips for loading software

Properly managing your software environment is key to avoiding conflicts and ensuring reproducibility. Here are some best practices:

- Avoid loading modules in your `.bashrc` file. Doing so can cause unexpected behavior, particularly in non-interactive environments like batch jobs or remote shells.
- Instead, load modules manually, from a separate script, or using module collections. This approach gives you more control and helps keep environments clean.
- Load required modules inside your job script. This ensures that your job runs with the expected software environment, regardless of your interactive shell settings.
- Be explicit about module versions. Short names like `gcc` will load the system default (e.g., `gcc/12.3`), which may change in the future. Specify full versions (e.g., `gcc/13.3`) for long-term reproducibility.

- Resolve dependencies with `module spider`. Some modules depend on others. Use `module spider <module-name>` to discover which modules are required and how to load them in the correct order. For more, see Sub-command spider.

## Using commercial software

You may be able to use commercial software on Trillium, but there are a few important considerations:

- Bring your own license. You can use commercial software on Trillium if you have a valid license. If the software requires a license server, you can connect to it securely using SSH tunnelling.
- We do not provide user-specific licenses. Due to the large and diverse user base, we cannot provide licenses for individual or specialized commercial packages.
- Some widely useful commercial tools are available system-wide, such as compilers (Intel), math libraries (MKL), debuggers (DDT).
- We're here to help. If you have a valid license and need help installing commercial software, feel free to contact us, we'll assist where possible.

# Testing and debugging

Before submitting your job to the cluster, it's important to test your code to ensure correctness and determine the resources it requires.

- **Lightweight tests** can be run directly on the login nodes. As a rule of thumb, these should:
  - Run in under a few minutes
  - Use no more than 1–2 GB of memory
  - Use only 1–4 CPU cores
  - Use at most 1 GPU
- You can also run the parallel ARM DDT debugger on the login nodes after loading it with `module load ddt-cpu` or `module load ddt-gpu`
- For tests that exceed login node limits or require dedicated resources, request an interactive debug job using the `debugjob` command on a login node:

```
$ debugjob
```

When run from a CPU login node, this command gives you an interactive shell on a CPU compute session for 1-hour. When running the debugjob command from the GPU login node, you get an interactive session with 1 GPU on a (shared) GPU compute node for two hours. A few variations of this command that you can use to request more resources for an interactive session, are given in the next table. Note that the more resources you request, the shorter the allowed walltime is (this helps makes sure that interactive session almost always start right away).

| Command | Subcluster | Number of nodes | Number of CPU cores | Number of GPUs | Memory | Walltime limit |
|---|---|---|---|---|---|---|
| debugjob | CPU | 1 | 192 | 0 | 755GiB | 60 minutes |
| debugjob 2 | CPU | 2 | 384 | 0 | 2x755GiB | 30 minutes |
| debugjob<br>debugjob -g 1 | GPU | 1/4 | 24 | 1 | 188GiB | 120 minutes |
| debugjob 1<br>debugjob -g 4 | GPU | 1 | 96 | 4 | 755GiB | 30 minutes |
| debugjob 2<br>debugjob -g 8 | GPU | 2 | 192 | 8 | 2x755GiB | 15 minutes |

The shell environment in a debugjob will be similar to the environment you get when you have just logged in: only standard modules loaded, no internet access, no write access to the home and project file systems, and no job submissions. By the way, if you want the session to inherit the modules that you had loaded before issuing the debugjob command, you can add "`--export=ALL`" as the first option to debugjob.

- If your test job requires more time than allowed by `debugjob`, you can request an interactive session from the regular queue using `salloc`. For CPU test jobs, the command would be as follows:

```
$ salloc --export=NONE --nodes=N --time=M:00:00 [--ngpus-per-node=G] [--x11]
```

where

- `N` is the number of nodes
- `M` is the number of hours the job should run
- `G` is the number of GPUs per node (when applicable).
- `--x11` is required for graphical applications (e.g., when using ARM DDT), but otherwise optional.

**Note:** Jobs submitted with `salloc` may take longer to start than with debugjob and count towards your allocation.

# Submitting jobs to the scheduler

Once you have compiled and tested your code or workflow on the Trillium login nodes and confirmed that it behaves correctly, you are ready to submit jobs to the cluster. These jobs will run on Trillium's compute nodes, and their execution is managed by the scheduler.

Trillium uses SLURM as its job scheduler. More advanced details of how to interact with the scheduler can be found on the **Slurm page**.

To submit a job, use the `sbatch` command on a login node:

```
$ sbatch jobscript.sh
```

CPU compute jobs need to be submitted from the CPU login nodes, while GPU compute nodes must be submitted from the GPU login node. In both cases, the command is the same, but the options inside the jobscript will have to be different (see below).

The sbatch command places your job into the queue. The job script should contain lines starting with `#SBATCH` that specify the resources that this script will need (the most common options will be given below). SLURM will begin execution of this script on compute nodes when your job is at the top of the priority queue and these resources are available.

The priority of a job in the queue depends on requested resources, time spent in the queue, recent past usage, as well as on the SLURM account under which the job was submitted. SLURM accounts correspond to **Resource Allocation Projects**, or RAPs:

- Each PI has at least one RAP, the RAS or default RAP. Users sponsored by that PI have access to the corresponding SLURM account, whose name starts with `def-`.
- PIs that have a RAC allocation have an additional RAC RAP, to which they can add users. The names of corresponding SLURM accounts typically start with `rrg-` or `rpp-`. Note that RACs are bound to a system, e.g. a RAC for Nibi cannot be used on Trillium.

## Trillium specific restrictions

Because Trillium is designed as a system for large parallel jobs, there are some differences with the General Purpose clusters **Fir**, **Nibi**, **Narval**, and **Rorqual**, which we will now discuss.

### Job output must be written to the scratch file system

The scratch file system is a fast parallel file system that you should use for writing out data during jobs. This is enforced by having the home and project directories only available for reading on the compute nodes.

In addition to making sure your application writes to scratch, in most cases, you should also submit your jobs from your `$SCRATCH` directory (i.e. not `$HOME` or `$PROJECT`). The default location for the output files of SLURM are in the directory from which you submit, so if that is not in scratch, the output files would not be written.

### Default scheduler account

Jobs will run under your group's RAC allocation, or if one is not available, under a RAS allocation. You can control this explicitly by specifying the account with the `--account=ACCOUNT_NAME` option in your job script or submission command. For users with multiple allocations, specifying the account name is highly recommended.

### No job submission from jobs

Jobs cannot be submitted from compute nodes (nor datamover nodes). This prevents accidentally spawning many jobs, overloading the scheduler, and overloading the backup process.

### Whole node or whole gpu scheduling

It is not possible to request a certain number of core on Trillium. On the CPU subcluster, all jobs must use full nodes. That means the minimum size of a CPU job has 192 cores are its disposal which you must use effectively. If you are running serial or low-core-count jobs you must still use all 192 cores on the node by bundling multiple independent tasks in one job script. For examples, see **GNU Parallel** and **this section of the META-Farm advanced page**.

If your job underutilizes the cores, our support team may reach out to assist you in optimizing your workflow, or you can **contact us (mailto:trillium@tech.alliancecan.ca)** to get assistance.

On the GPU subcluster, each node contains 4 GPUs. The scheduler allows you to request either a whole number of nodes, or a single GPU. The latter amounts to a quarter node, with 24 cores and about 188GiB of RAM. It is important to use the GPU efficiently. Trillium does not support MIG as on the other clusters (MIG allows you to schedule a fraction of a GPU), but you can use **Hyper-Q / MPS** within your jobs.

### Memory requests are ignored

Memory requests are ignored. Your CPU jobs always receive `N × 768GB` of RAM, where `N` is the number of nodes and 768GB is the amount of memory on each node. Your GPU full-node jobs get the same amount of memory, while single-GPU jobs get 1/4 of the memory, i.e., 188GiB.

## Common options for job script

The following options are commonly used:

| option | short option | meaning | notes |
|---|---|---|---|
| `--nodes` | `-N` | number of nodes | Recommended to always include this |
| `--ntasks-per-node` | | number of tasks for srun/mpirun to launch per node | Prefer this over `--ntasks` |
| `--ntasks` | `-n` | number of tasks for srun/mpirun to launch | |
| `--cpus-per-task` | `-c` | number of cores per task; | Typically for (OpenMP) threads |
| `--time` | `-t` | duration of the job | |
| `--job-name` | `-J` | specify a name for the job | |
| `--output` | `-o` | file to redirect standard ouput to | Can be a pattern using e.g. %j for the jobid. |
| `--mail-type` | | when to send email (e.g. BEGIN, END, FAIL, ALL) | |
| `--gpus-per-node` | | number of gpus to use on each node | Either 1 or 4 is allowed on the GPU subcluster |
| `--partition` | `-p` | partition to submit to | See below for available partitions |
| `--account` | `-A` | slurm account to use | For many users, this is automatic on Trillium |
| `--mem` | | amount of memory requested | Ignored on Trillium, you get all the memory |

These options should be put in separate comment lines at the top of the job script (but after `#!/bin/bash`), prefixed with `#SBATCH`. They can also be used as command line options for `salloc`. Some examples of job scripts are given below.

More options and details can be found on the **Running jobs** page and in the **SLURM documentation (https://slurm.schedmd.com/sbatch.html)**.

# Submitting jobs on the CPU subcluster

## Partitions and limits

There are limits to the size and duration of your jobs, the number of jobs you can run, and the number of jobs you can have queued. It matters whether a user is part of a group with a RAC allocation (e.g. an RRG or RPP) or not. It also matters in which "partition" the job runs. "Partitions" are SLURM-speak for use cases. You specify the partition with the `-p` parameter to `sbatch` or `salloc`, but if you do not specify one, your job will run in the `compute` partition, which is the most common case.

| Usage | Partition | Limit on Running jobs | Limit on Submitted jobs (incl. running) | Min. size of jobs | Max. size of jobs | Min. walltime | Max. walltime |
|---|---|---|---|---|---|---|---|
| Compute jobs | compute | 150 | 500 | 1 node (192 cores) | default: 10 nodes (1920 cores)<br>with allocation: 128 nodes (24576 cores)* | 15 minutes | 24 hours |
| Testing or troubleshooting | debug | 1 | 1 | 1 node (192 cores) | 2 nodes (384 cores) | N/A | 1 hour |

* This is a safe-guard, if your rrg involves running larger jobs, let us know.

Even if you respect these limits, your jobs will still have to wait in the queue. The waiting time depends on many factors such as your group's allocation amount, how much allocation has been used in the recent past, the number of requested nodes and walltime, and how many other jobs are waiting in the queue.

## Example: MPI job

```
#!/bin/bash
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=192
```

```
#SBATCH --time=01:00:00
#SBATCH --job-name=mpi_job
#SBATCH --output=mpi_output_%j.txt
#SBATCH --mail-type=FAIL

cd $SLURM_SUBMIT_DIR

module load StdEnv/2023
module load gcc/12.3
module load openmpi/4.1.5

source /scinet/vast/etc/vastpreload-openmpi.bash # important if doing MPI-IO

mpirun ./mpi_example
```

Submit this script from a CPU login node while in your `$SCRATCH` directory with the command:

```
$ sbatch mpi_job.sh
```

- First line indicates that this is a bash script.
- Lines starting with #SBATCH go to SLURM.
- `sbatch` reads these lines as a job request (which it gives the name `mpi_job`).
- In this case, SLURM looks for 2 nodes each running 192 tasks, for 1 hour.
- Once it finds such nodes, it runs the script, which does the following:
  - Change to the submission directory;
  - Loads modules;
  - Preloads a library tuning MPI-IO for the VAST file system; change this to source /scinet/vast/etc/vastpreload-intelmpi.bash if using intelmpi instead of openmpi.
  - Runs the `mpi_example` application (SLURM will inform `mpirun` or `srun` how many processes to run).

## Example: OpenMP job

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=192
#SBATCH --time=01:00:00
#SBATCH --job-name=openmp_job
#SBATCH --output=openmp_output_%j.txt
#SBATCH --mail-type=FAIL

cd $SLURM_SUBMIT_DIR

module load StdEnv/2023
module load gcc/12.3

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

./openmp_example
# or "srun ./openmp_example"
```

Submit this script from a CPU login node while in your `$SCRATCH` directory with the command:

```
$ sbatch openmp_job.sh
```

- First line indicates that this is a Bash script.
- Lines starting with #SBATCH are directives for SLURM.
- `sbatch` reads these lines as a job request (which it gives the name `openmp_job`).
- In this case, SLURM looks for one node with 192 CPUs for a single task running up to 192 OpenMP threads, for 1 hour.
- Once such a node is allocated, it runs the script:
  - Changes to the submission directory;

- Loads the required modules;
- Sets `OMP_NUM_THREADS` based on SLURM's CPU allocation;
- Runs the `openmp_example` application.

### Example: hybrid MPI/OpenMP job

```
#!/bin/bash
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=48
#SBATCH --cpus-per-task=4
#SBATCH --time=01:00:00
#SBATCH --job-name=hybrid_job
#SBATCH --output=hybrid_output_%j.txt
#SBATCH --mail-type=FAIL

cd $SLURM_SUBMIT_DIR

module load StdEnv/2023
module load gcc/12.3
module load openmpi/4.1.5

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OMP_PLACES=cores
export OMP_PROC_BIND=true

export CORES_PER_L3CACHE=8
export RANKS_PER_L3CACHE=$(( $CORES_PER_L3CACHE / $OMP_NUM_THREADS ))  # this works up to 8 threads

source /scinet/vast/etc/vastpreload-openmpi.bash # important if doing MPI-IO

mpirun --bind-to core --map-by ppr:$RANKS_PER_L3CACHE:l3cache:pe=$OMP_NUM_THREADS ./hybrid_example
```

Submit this script from a CPU login node while in your `$SCRATCH` directory with the command:

```
$ sbatch hybrid_job.sh
```

- First line indicates that this is a bash script.
- Lines starting with #SBATCH go to SLURM.
- `sbatch` reads these lines as a job request (which it gives the name `hybrid_job`).
- In this case, SLURM looks for 2 nodes each running 48 tasks, each with 4 threads for 1 hour.
- Once it finds such a node, it runs the script:
  - Change to the submission directory;
  - Loads modules;
  - Preloads a library tuning MPI-IO for the VAST file system; change this to source /scinet/vast/etc/vastpreload-intelmpi.bash if using intelmpi instead of openmpi.
  - Runs the `hybrid_example` application. While SLURM will inform `mpirun` how many processes to run, it needs help to spread the processes and threads evenly over the cores. The --map-by option solves this.
    (for more than 8 and at most 24 threads per process, change 'l3cache' to 'numa' and for more than 24, change it to 'socket').

# Submitting jobs for the GPU subcluster

### Partitions and limits

As with the CPU subcluster, there are limits to the size and duration of your jobs, the number of jobs you can run, and the number of jobs you can have queued, and whether a user is part of a group with a RAC allocation or not. There are more partitions for this subcluster than for the CPU subcluster to support scheduling by GPU instead of by node (each node has 4 GPUs).

On Trillium, you are only allowed to request exactly 1 GPU or a multiple of 4 GPUs. You cannot request --gpus-per-node=2 or 3, nor can you use NVIDIA's MIG technology to allocate a subdivision of a GPU. Inside a job, you can use NVIDIA's Multi-Process Service (MPS) to share a GPU among processes running on the same job.

- For single-GPU jobs, use `--gpus-per-node=1`.
- For whole-node GPU job, use `--gpus-per-node=4`.

| Usage | Partition | Limit on Running jobs | Limit on Submitted jobs (incl. running) | Min. size of jobs | Max. size of jobs | Min. walltime | Max. walltime |
|---|---|---|---|---|---|---|---|
| GPU compute jobs | compute | 150 | 500 | 1/4 node (24 cores / 1GPU) | default: 5 nodes (480 cores/20 GPUs) with allocation: 25 nodes (2400 cores/100 GPUs) | 15 minutes | 24 hours |
| Testing GPU jobs | debug | 1 | 1 | 1/4 node (24 cores / 1 GPU) | 2 nodes (192 cores/ 8 GPUs) | N/A | 2 hours (1 GPU) - 30 minutes (8 GPUs) |

Even if you respect these limits, your jobs will still have to wait in the queue. The waiting time depends on many factors such as your group's allocation amount, how much allocation has been used in the recent past, the number of requested nodes and walltime, and how many other jobs are waiting in the queue.

## Example: Single-GPU Job

```
#!/bin/bash
#SBATCH --job-name=single_gpu_job         # Job name
#SBATCH --output=single_gpu_job_%j.out    # Output file (%j = job ID)
#SBATCH --nodes=1                         # Request 1 node
#SBATCH --gpus-per-node=1                 # Request 1 GPU
#SBATCH --time=00:30:00                   # Max runtime (30 minutes)

# Load modules
module load StdEnv/2023
module load cuda/12.6
module load python/3.11.5

# Activate Python environment (if applicable)
source ~/myenv/bin/activate

# Check GPU allocation
srun nvidia-smi

# Run your workload
srun python my_script.py
```

## Example: Whole-Node (4 GPUs) Job

```
#!/bin/bash
#SBATCH --job-name=whole_node_gpu_job
#SBATCH --output=whole_node_gpu_job_%j.out
#SBATCH --nodes=1
#SBATCH --gpus-per-node=4
#SBATCH --time=02:00:00

module load StdEnv/2023
module load cuda/12.6
module load python/3.11.5

# Activate Python environment (if applicable)
source ~/myenv/bin/activate
```

```
srun python my_distributed_script.py
```

### Example: Multi-Node GPU Job

```
#!/bin/bash
#SBATCH --job-name=multi_node_gpu_job
#SBATCH --output=multi_node_gpu_job_%j.out
#SBATCH --nodes=2                        # Request 2 full nodes
#SBATCH --gpus-per-node=4                # 4 GPUs per node (full node)
#SBATCH --time=04:00:00

module load StdEnv/2023
module load cuda/12.6
module load openmpi/4.1.5

# Check all GPUs allocated
srun nvidia-smi

# Activate Python environment (if applicable)
source ~/myenv/bin/activate

# Example: run a distributed training job with 8 GPUs (2 nodes × 4 GPUs)
srun python train_distributed.py
```

### Best Practices for GPU Jobs

- Do not use `--mem` — memory is fixed per GPU (192 GB) or per node (768 GB).
- Always specify node count, and `--gpus-per-node=4` for whole-node or multi-node jobs.
- Load only the modules you need — see Using modules.
- Be explicit with software versions for reproducibility (e.g., `cuda/12.6` rather than just `cuda`).
- Test on a single GPU before scaling to multiple GPUs or nodes.
- Monitor usage with `nvidia-smi` to ensure GPUs are fully utilized.

# Monitoring

## Monitoring the queue

Once your job is submitted to the queue, you can monitor its status and performance using the following SLURM commands:

- `squeue` shows all jobs in the queue. Use `squeue -u $USER` to view only your jobs.
- `squeue -j JOBID` shows the current status of a specific job. Alternatively, use `scontrol show job JOBID` for detailed information, including allocated nodes, resources, and job flags.
- `squeue --start -j JOBID` gives a rough estimate of when a pending job is expected to start. Note that this estimate is often inaccurate and can change depending on system load and priorities.
- `scancel JOBID` cancels a job you submitted.
- `jobperf JOBID` gives a live snapshot of the CPU and memory usage of your job while it is running.
- `sacct` shows information about your past jobs, including start time, run time, node usage, and exit status.

More details on monitoring jobs can be found on the **Slurm page**.

## Monitoring running and past jobs

Note that after your job has finished, it will be removed from the queue, so SLURM commands that query the queue like squeue and sacct will not find your job anymore.

Your past jobs and their resource usage can be inspected through the **my.SciNet (https://my.scinet.utoronto.ca)** portal. This portal saves information about all jobs, including performance data collected every two minutes while the job was running.

# Quick Reference for Common Commands

| Command | Description |
|---|---|
| sbatch <script> | Submit a batch job script |
| squeue [-u $USER] | View queued jobs (optionally for current user) |
| scancel <JOBID> | Cancel a job |
| sacct | View accounting data for recent past jobs |
| module load <module> | Load a software module |
| module list | List loaded modules |
| module avail | List available modules |
| module spider <module> | Search for modules and dependencies |
| debugjob [N] | Request a short debug job (on N nodes) |
| diskusage_report | Check storage quotas |
| jobperf <JOBID> | Monitor CPU and memory usage of a running job |
| nvidia-smi | Check GPU status (on GPU nodes) |

Retrieved from "https://docs.alliancecan.ca/mediawiki/index.php?title=Trillium_Quickstart&oldid=191222"

**This page was last edited on 16 September 2025, at 21:28.**